

Multiprocessor programming, DV2597/DV2606

Threads Programming (Chapter 3 and 6)

Håkan Grahn

(many slides by G. Karypis, T. Rauber and G. Rünger)

Topic overview

- Processes and threads
- Pthreads programming

Topic overview

- Processes and threads
- Pthreads programming

Processes and threads

- The programming of multicore processors is closely related to shared memory parallel programming and thread programming.
- **Threads** = Flows of computations of the same program, that are computed in parallel and that access common variables of the shared memory
- Example of current programming environments:
  - Pthreads, Java-Threads, OpenMP, and Win32

Processes

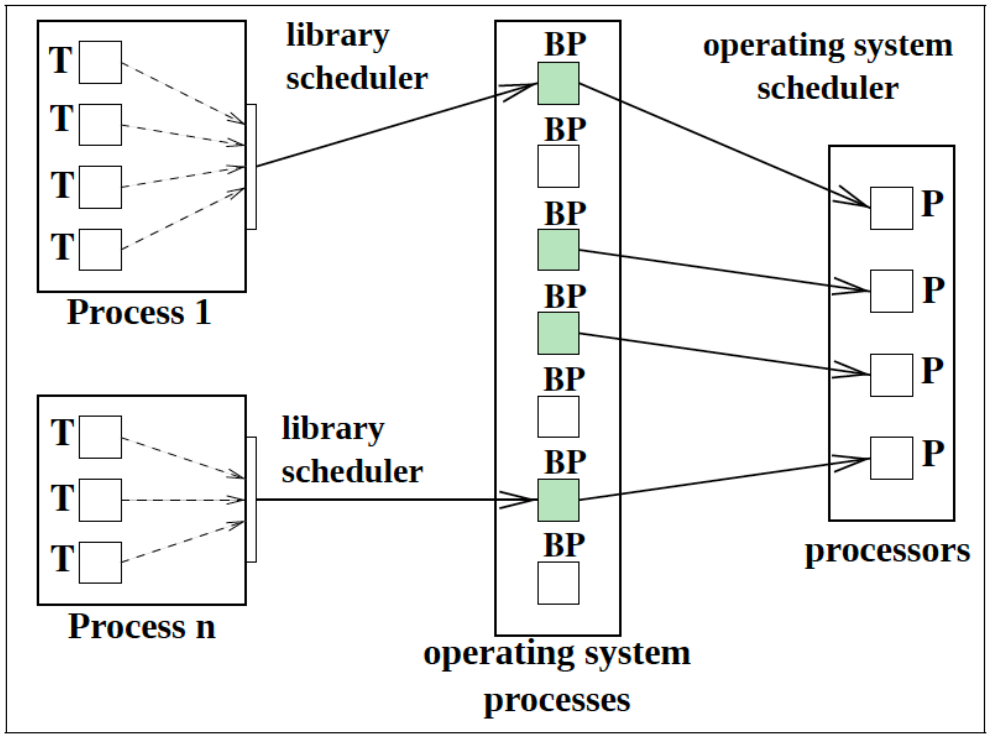
A **process** is a program in execution and includes the program code and all information (e.g. data on the runtime stack or on the heap, the content of the registers). In summary it includes all the data required for the execution

- Own address space
- Context switch: Change of state through process switching
  - Multitasking: Processes are executed with a the time slicing method
    $\rightsquigarrow$ Concurrent but not parallel
  - Multiprocessor-Systems: true parallelism
- Creation of a process: `fork`-call
  Child process: *Copy* of the address space, identical program after the `fork`-call
- Rating: Creation and management of processes is relatively time consuming

Threads

- Extension of the process model: A process includes several independent flows of computations (= Threads)
- Threads share the address space of their process
  $\rightsquigarrow$ Fast information exchange (compared to sockets on process-level)
- Fast creation of threads
- Execution on different processors or cores possible
  $\rightsquigarrow$ True parallelism
- Different thread levels:
  - User level threads: Fast thread switching; OS has no knowledge of threads $\rightsquigarrow$ CPU stops whole process (e.g. for I/O) no switch to an other thread possible
  - OS-level

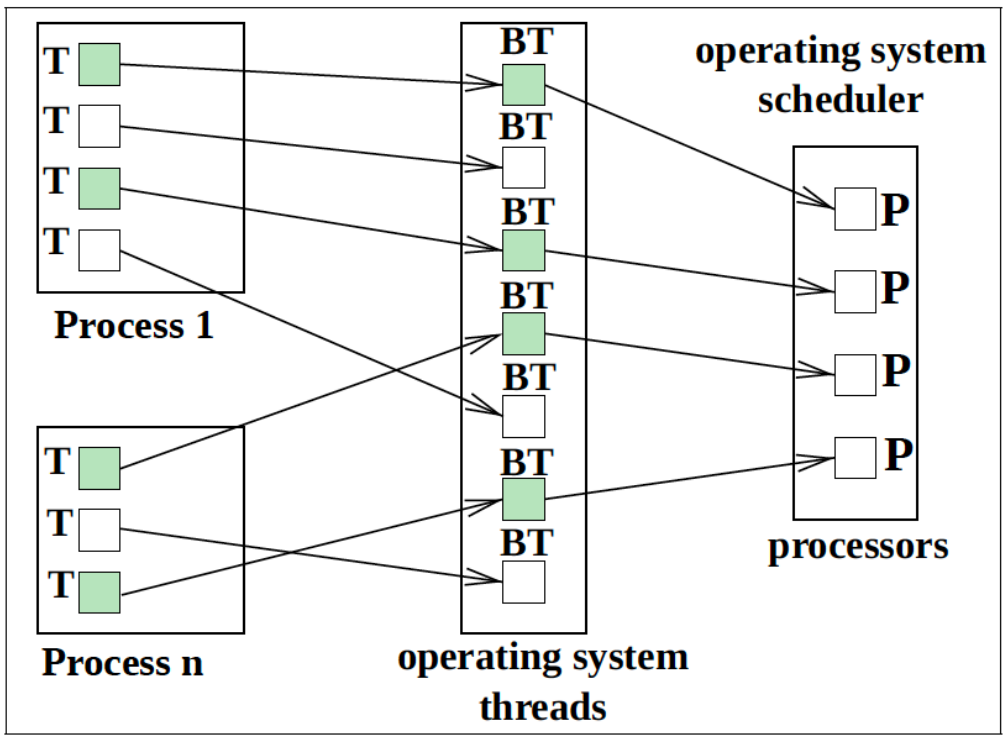
Thread execution models (1)



N:1-Mapping

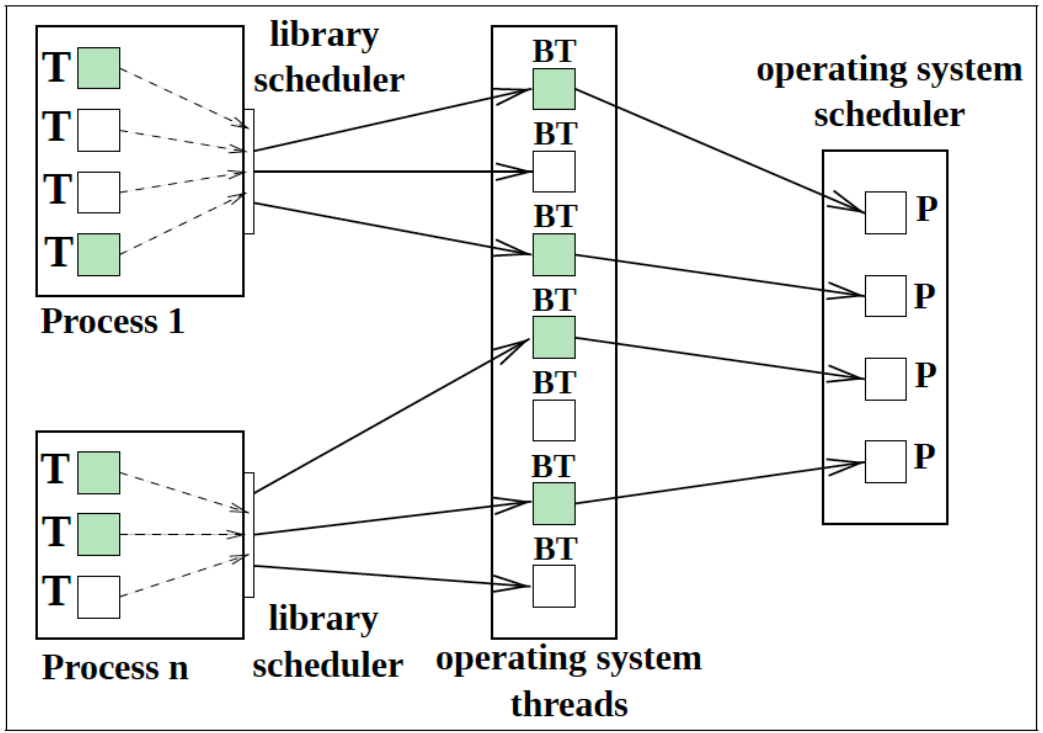
Thread execution models (2)

1:1-Mapping



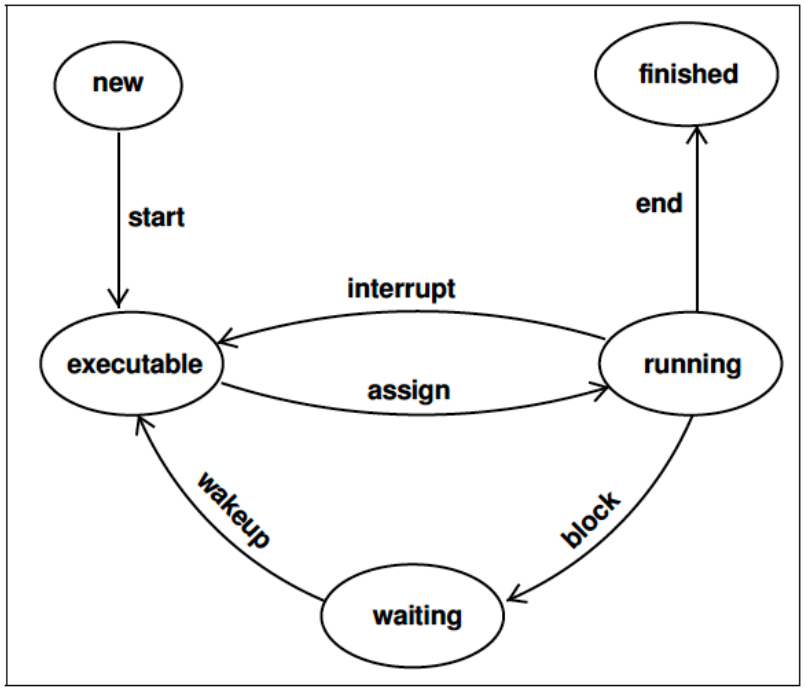
Thread execution models (3)

N:M-Mapping



Thread states

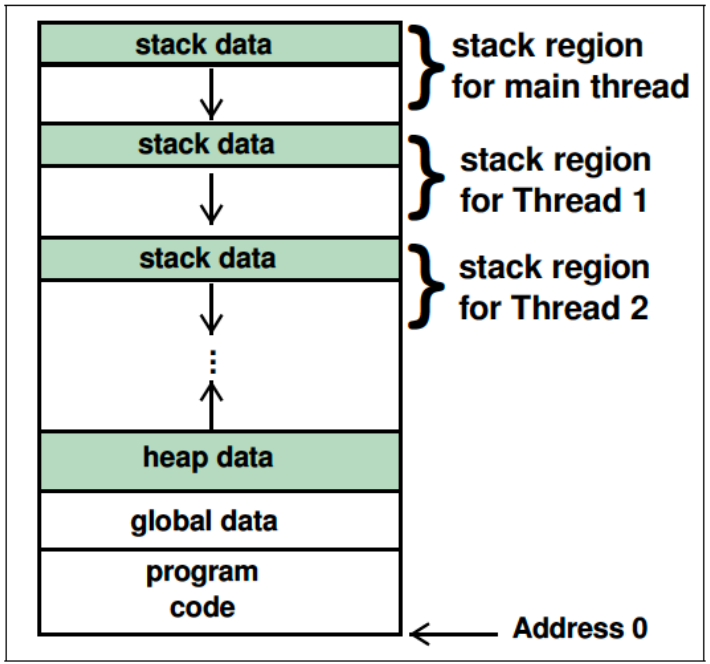
- New
- Executable
- Running
- Waiting
- Finished



Visibility of data

Run-time management for a program with several threads

- Private runtime stack per thread for local data
- Local data of a thread is invisible for other threads
- Run-time stack exists as long as the thread exists
- Special methods for return value passing required



Synchronization = Coordination of threads

- Getting a desired order of execution
- Designing the access to the shared memory

- The order of execution of computations causes a special state of the shared memory, i.e. assignment of values to the shared variable
- Avoidance of undesired effects during the parallel access to a shared variable

Effects of cooperation and coordination of threads can be different between concurrency or parallelism

Synchronization terminology

- Coordination of flows of computations - **Barrier-Synchronization**
- Race conditions
- Critical section
- Lock-Mechanism
- Condition-Variables
- Semaphor-Mechanism
- Monitor

Barrier-Synchronization:
All participating threads wait for each other and no thread executes an instruction located behind the synchronization instruction.

Critical regions – problem and solution

- **Race conditions:**
  The result of the parallel execution of a program fragment by several threads depends on the relative execution speed of the threads:
  - Executing the program fragment first by Thread T_1 and then by Thread T_2 will compute a different result than if first Thread T_2 executes the fragment and afterwards Thread T_1.

  $\rightsquigarrow$ **non deterministic behavior**
- **Critical section:**
  Program fragment with accesses to shared variables that may also be accessed concurrently by different threads
  - A correct execution order can be achieved by using a **mutual exclusion**, to ensure that one thread can access the variable of the critical section exclusively.
  - Methods: Lock-Synchronization, Condition-Variables.

Locks

- **Lock variable** s (or mutex variable)
- Function *lock*(s):
  Before entering the critical region the thread executes *lock*(s) for locking the lock-variable s;
- Function *unlock*(s):
  After leaving the critical region *unlock*(s) has to be called

⇝ Only when *each* processor uses this convention, race conditions are avoided

Effects of lock(s) for Thread T_1:

- If no other thread has locked s:
  Assignment of s to T_1
- Another thread T_2 locks the lock-variable s:
  Thread T_1 blocks until Thread T_2 releases s by calling `unlock(s)`

Realization in Pthreads, Java-Threads or OpenMP

Condition variables

- With **condition variables** a Thread T_1 blocks until a specific condition occurs.
- Waking up a thread can only be done by another Thread T_2.
- After waking up, thread T_1 tests the condition again.
- For avoiding race conditions, additional lock variables are used

Semaphore synchronization

Semaphore-Synchronization:

- A semaphore is a structure that includes an integer counter s that can be accessed by two atomic operations
  - $P(s)$ (or $wait(s)$) waits until the value of s is greater than 0, decrements the value of s by 1 afterwards and allows the execution of the following statements.
  - $V(s)$ (or $signal(s)$) increments the value of s by 1.
- Typical pattern:

```
wait(s)
critical section
signal(s)
```

Monitor

Monitor:

- A monitor is a language construct that combines data and operations accessing that data together into a structure
- Access to monitor data only using monitor operations
- At each time the executing of only **one** monitor operation is allowed

Topic overview

- Processes and threads
- Pthreads programming